Toronto Metropolitan University

Computer Science
Graduate Studies

Course Number	CPS 8319
Course Title	Reinforcement Learning
Semester/Year	
Section Number	
Group Number	

Assignment No.2

` Student Name	Student ID (xxxx1234)	
John Cook-Martin		
Pari Qarehdaghi	4602	

Assignment Title	Question 3, Dynamic Programming
Submission Date	2026-02-18
Due Date	2026-02-18

Contents

Report 3 — Dynamic Programming Methods for Gridworld 4

Introduction 4

Problem Setup 4

Algorithms Used 4

Value Iteration Algorithm (VIA) 4

Policy Iteration Algorithm (PIA) 5

Results 5

Value Iteration Results 5

Policy Iteration Results 6

Observed properties: 8

Comparison Between Value Iteration and Policy Iteration 8

Conclusion 8

References (MLA) 9

Figure 1:Final value function obtained using Value Iteration (k = 5) 6

Figure 2: Policy Iteration result showing final policy evaluation iterations (k = 8) 7

Figure 3: Policy Iteration result showing total policy evaluation iterations (k = 79) 7

Report 3 — Dynamic Programming Methods for Grid world

Introduction

In this question, we study Dynamic Programming (DP) methods for solving a finite Markov Decision Process (MDP) using a simple 4×4 Grid World environment. The goal is to evaluate and compare two classical DP algorithms:

- Value Iteration Algorithm (VIA)
- Policy Iteration Algorithm (PIA)

Both methods compute the optimal value function and the corresponding optimal policy, assuming a fully known environment model. The Grid World contains terminal states, deterministic transitions, and a constant step cost, making it a suitable testbed for understanding the behavior and convergence properties of DP algorithms.

Problem Setup

The environment consists of a 4×4 grid (16 states), indexed from 0 to 15.
The top-left and bottom-right states are terminal states, with value fixed at zero.

- State space: 16 discrete states
- Action space: Up, Down, Left, Right (with boundary constraints)
- Transition model: Deterministic
- Reward model:
  - Each non-terminal move incurs a negative reward (step cost)
  - Terminal states return zero reward
- Discount factor: $\gamma = 0.95$

Because the transition probabilities are deterministic, the Bellman updates simplify significantly and allow direct application of standard DP equations.

Algorithms Used

Value Iteration Algorithm (VIA)

Value Iteration repeatedly applies the Bellman optimality update to all states until convergence:

$$V_{k+1}(s) = \max_{a} \sum_{s'} p(s' \mid s, a)[r(s, a, s') + \gamma V_k(s')]$$

At each iteration:

1. The value of every state is updated using the maximum expected return over all actions.
2. The algorithm stops when the maximum change in value across states falls below a small threshold.
3. Once converged, the optimal policy is extracted by selecting the action that maximizes the Bellman expression.

Policy Iteration Algorithm (PIA)

Policy Iteration alternates between two phases:

- **Policy Evaluation**

Given a fixed policy π, the value function is updated iteratively until convergence:

$$V^{\pi}(s) = \sum_{s'} p(s' \mid s, \pi(s))[r(s, \pi(s), s') + \gamma V^{\pi}(s')]$$

- **Policy Improvement**

The policy is updated greedily with respect to the evaluated value function:

$$\pi_{\text{new}}(s) = \arg\max_{a} \sum_{s'} p(s' \mid s, a)[r(s, a, s') + \gamma V^{\pi}(s')]$$

These two steps repeat until the policy no longer changes, indicating convergence to the optimal policy.

Results

Value Iteration Results

The Value Iteration algorithm converged after 5 iterations.

Observed properties:

- Values become increasingly negative as states move farther from terminal states.
- The resulting value function is symmetric with respect to the grid geometry.

- Convergence is relatively fast but requires multiple full sweeps over the state space.

```
GRID 16      k = 5      i = 5
------------------------

VALUES:
**: 0.00  1: -1.17  2: -2.29  3: -3.45
 4: -1.17  5: -2.23  6: -3.32  7: -2.29
 8: -2.29  9: -3.32 10: -2.23 11: -1.17
12: -3.45 13: -2.29 14: -1.17 **: 0.00
```

Figure 1:Final value function obtained using Value Iteration (k = 5)

Policy Iteration Results

Policy Iteration converged in 2 policy improvement iterations, but required multiple internal policy-evaluation sweeps.

Two iteration counts are reported:

- k (final): Number of iterations in the *last* policy evaluation
- k (total): Total number of policy evaluation iterations across all policy updates

Final policy evaluation only:

- k = 8
- i = 2 (policy improvements)

```
GRID 16      k = 8      i = 2
-------------------------

VALUES:
**: 0.00  1: -1.17  2: -2.26  3: -3.29
 4: -1.17  5: -2.20  6: -3.18  7: -2.26
 8: -2.26  9: -3.18 10: -2.20 11: -1.17
12: -3.29 13: -2.26 14: -1.17 **: 0.00
ACTIONS
[TERM] [L   ] [L   ] [BL  ]
[T   ] [TL  ] [BL  ] [B   ]
[T   ] [TR  ] [BR  ] [B   ]
[TR  ] [R   ] [R   ] [TERM]
```

Figure 2: Policy Iteration result showing final policy evaluation iterations (k = 8)

Total policy evaluation iterations:

- k = 79
- i = 2

```
GRID 16      k = 79      i = 2
-------------------------

VALUES:
**: 0.00  1: -1.17  2: -2.26  3: -3.29
 4: -1.17  5: -2.20  6: -3.18  7: -2.26
 8: -2.26  9: -3.18 10: -2.20 11: -1.17
12: -3.29 13: -2.26 14: -1.17 **: 0.00
ACTIONS
[TERM] [L   ] [L   ] [BL  ]
[T   ] [TL  ] [BL  ] [B   ]
[T   ] [TR  ] [BR  ] [B   ]
[TR  ] [R   ] [R   ] [TERM]
```

Figure 3: Policy Iteration result showing total policy evaluation iterations (k = 79)

Observed properties:

- Policy Iteration converges in far fewer policy improvement steps than Value Iteration.
- Each policy evaluation phase is computationally more expensive.
- The final value function and policy match those obtained by Value Iteration.

Comparison Between Value Iteration and Policy Iteration

Value Iteration and Policy Iteration are both dynamic programming methods that solve Markov Decision Processes using Bellman equations, but they differ in how value updates and policy improvements are performed.

- Value Iteration applies the Bellman optimality update directly to the value function and derives the optimal policy only after convergence.
- Policy Iteration explicitly alternates between policy evaluation using the Bellman expectation equation and policy improvement via a greedy update.
- Value Iteration generally requires more iterations with simpler updates, while Policy Iteration converges in fewer policy updates but with higher computational cost per iteration.
- Despite these differences, both algorithms converge to the same optimal value function and optimal policy, as observed in the experimental results.

Conclusion

In this question, we successfully applied Value Iteration and Policy Iteration to a 4×4 Grid World MDP. Both algorithms converged to the same optimal solution, validating the correctness of the implementations.

While Value Iteration required fewer total computations per iteration, Policy Iteration converged in fewer policy updates. This highlights the classic trade-off between the two methods. Overall, Policy Iteration demonstrated faster convergence in terms of decision quality, whereas Value Iteration offered a simpler and more uniform update process.

References (MLA)

Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press.